Hunter Drake
Ian Wyse
Nelson Rodriguez-Ortiz
Paul Globisch

ECE 540: Now in 3D!

Overview

This project concerns the creation of a 2.5D + 3D rendering pipeline for the RVfpga SoC. The goal is to render a basic scene, using 2.5D for the background and walls, with 3D for an object. The demo output of this project would be showing movement through the scene using a keyboard, the Nexys A7 board, and a monitor.

Work Distribution

- Hunter Drake
  - 3D Graphics Card Wishbone Bus Register Interface
  - 3D Graphics Card Drivers
  - General RTL
- Ian Wyse
  - 2.5D basic functionality
  - Underlying frame data functionality for BRAM
- Nelson Rodriguez-Ortiz
  - GPU FSM for 3D rendering
  - BRAM backpressure mechanism and clock crossing handling between Wishbone clock/GPU clk
- Paul Globisch
  - 3D rendering software
  - Hardware/interface integration as needed

Committed Goals

Our committed goals are to create a basic 2.5D environment that the user can explore, similar to those created for games like Doom and Wolfenstein in the early 1990s. The plan for executing this goal is laid out in the block diagram (fig. 1), and explained below. The environment will consist of a 2D grid filled with walls and open spaces, which will be viewed by the user as a 3D environment. Modeling will be done using the digital differential analyzer algorithm with fixed point math. The algorithm determines the distance between the user and the nearest wall for every pixel column in the output, and then uses this to calculate the height of the wall at that location, in pixels.

Each column height will be sent to a custom graphics accelerator module via the wishbone bus, which will write the correct pixel colors for that column out to a frame buffer

stored in BRAM. This module will operate at 100MHz in order to ensure that writes complete before any new data arrives.

The VGA controller module will read data from the frame buffer and output it to the monitor correctly aligned with synchronization signals from the display timing module. User input will be read from the arrow keys on a keyboard via a custom keyboard controller module.

In a separate mode, we would also render a single, true 3D, mesh in the scene (most likely a rotating cube). This would involve defining a mesh out of vertices and triangles, applying transformations and then projecting it to 2D screen space. The GPU would be extended to rasterize the triangles (either a separate module or a “mode” where it can draw a line between 2 points for scene, and fill for 3 point triangles). This mesh would be flat-shaded with defined colors and draw-order (Painter's Algorithm). Since the computations software side for this would be minimal compared to raycasting, it would likely not affect software performance. The block diagram for the true 3D render would look similar to Figure 1, with the addition of the 3d mesh calculations to the C-program, an extension to the GPU for arbitrary line drawing or triangle rasterization.

Stretch Goals

Our primary stretch goal is to integrate the 3D rendered mesh in the 2.5D scene. This would involve determining a draw order for the walls and mesh, as well as figuring out a good method for drawing them into the same frame buffer. This is a stretch goal due to the potential for unforeseen issues, but it may also be easily achievable if we get lucky.

An additional 2 stretch goals would be texture mapping for the 2.5D scene and basic normals/lighting calculations for the 3D mesh. The 2.5D texture mapping would likely be done in hardware. The normals and lighting calculations for the 3D mesh would likely be primarily done in software.

Contingency Plan

One of the major risks we face is an unforeseen performance bottleneck. Given this is an educational demo, we have a fair amount of flexibility in the final framerate, but there is still the risk of it being “unwatchable”. There are several options for mitigating this if it becomes a concern:

- We can reduce the target resolution further to cut down on the total compute required.
- We can reduce the amount of raycasts in the application, and opt for interpolation to fill in the gaps.
- Lookup tables can be added for many calculations.
- Many other tricks like partial redraws and hardcoded backgrounds.
- Some elements of the raycasting algorithm could potentially be moved into custom hardware blocks, where a higher frequency could be used.
- If the software is determined to be the bottleneck, a nuclear option would be to use a Microblaze core instead of RVfpga. The Microblaze core is capable of running at at least

100 MHz, potentially speeding up the software dramatically. This option is undesirable as it would require reworking modules for AXI and a large overall hardware design change.

Diagrams

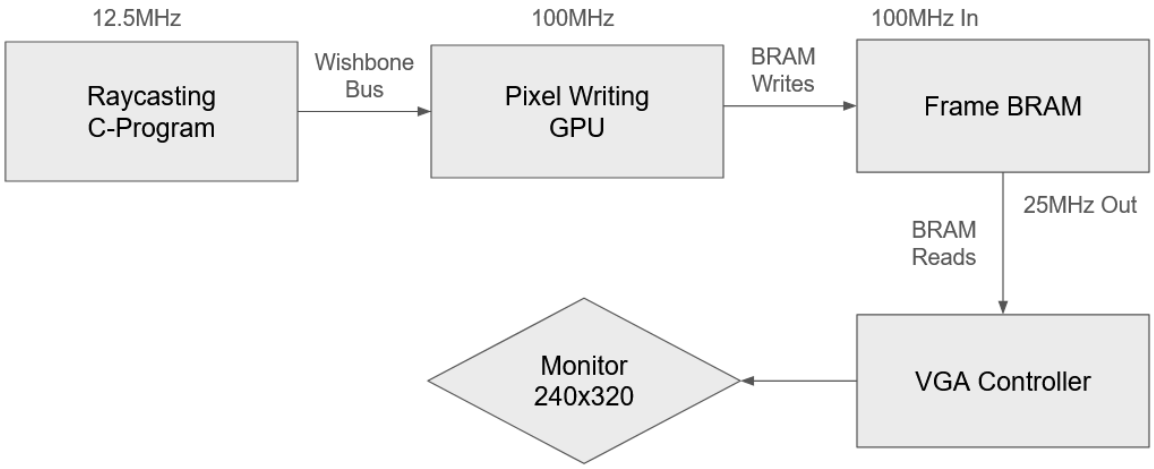

Figure 1. Basic 2.5D raycasting

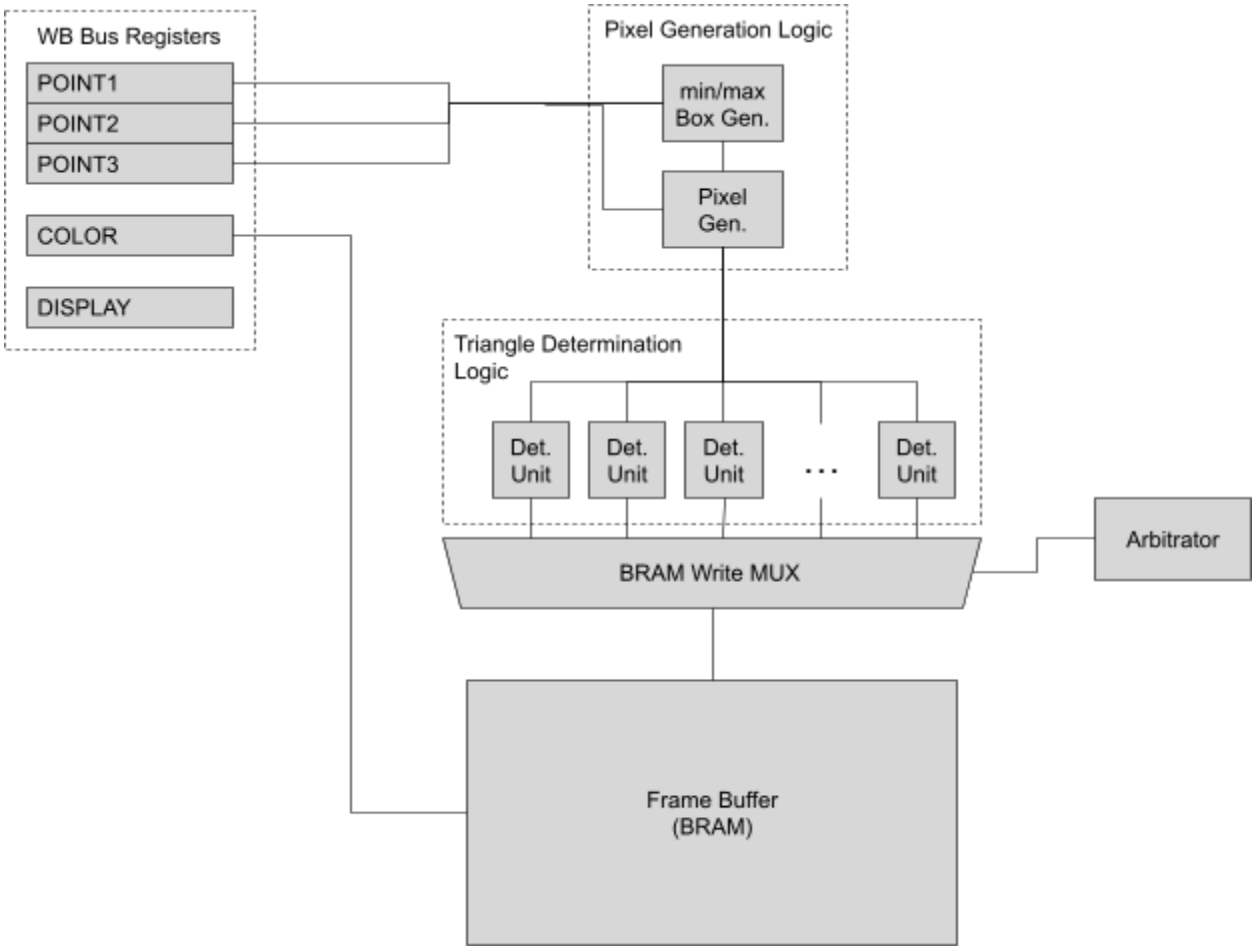

Figure 2. Full 3D Rendering